Национальный исследовательский университет «Высшая школа экономики»

Факультет компьютерных наук

Департамент программной инженерии

Электронный аукцион

Отчёт по индивидуальному заданию № 3

по дисциплине «Архитектура компьютера и операционные системы»

Разработка имитационной программы
с использованием взаимодействующих процессов

Вариант 37

Выполнил: Рыжов Степан Игоревич

Группа: БПИ252

Москва 2026

1. Условие задачи и устройство модели

Требуется разработать консольное приложение на C для Linux, моделирующее электронный аукцион. Несколько лотов продаются последовательно или параллельно. У лота есть начальная цена, минимальный шаг и срок закрытия. Участники имеют ограниченные бюджеты, стратегии и задержки ставок. Средства по ставкам резервируются; возможны продление торгов и отказ победителя.

Выполнены все четыре реализации, предусмотренные условием для уровня выше 8 баллов.

Входные параметры: число участников и лотов, цены и шаги, бюджеты, длительности торгов, продление, стратегии, задержки, число одновременно открытых лотов, правила резервирования и отказа.

Процессы и сценарий

Сущность	Число	Поведение
Аукционист	1	Открывает лоты, принимает и отклоняет ставки, ведёт расчёты и статистику.
Участник	N	Выбирает открытый лот, готовит ставку и отправляет её после задержки.
Лот	M	Следит за сроком с учётом продлений и запрашивает закрытие.

Каждая сущность из таблицы представлена отдельным процессом. Всего работают N + M + 1 процессов модели, максимум 65.

Сценарий взаимодействия: аукционист открывает лоты, участники передают ставки, процессы лотов запрашивают закрытие по сроку. Аукционист обрабатывает эти события, проводит расчёты и открывает следующие лоты. Способы передачи событий описаны в разделе 3.

Участник выбирает первый открытый лот, где он не лидирует и не исключён после отказа. Одновременно готовится одна ставка. Её сумма фиксируется при подготовке: текущая цена плюс один или два шага.

Аукционист проверяет состояние, срок, раунд, минимальный шаг и доступные деньги. Собственный резерв в этом лоте можно использовать для новой ставки:

```
доступно = бюджет - все резервы + собственный резерв в этом лоте
```

При -r 0 резерв прежнего лидера освобождается сразу; при -r 1 сохраняется до закрытия раунда. Принятая ставка продлевает срок до текущего времени плюс интервал продления, если это позже прежнего срока. При закрытии резервы освобождаются, победитель платит. При разрешённом отказе лот открывается заново с исходной ценой, отказавшийся исключается из этого лота. Без ставок лот остаётся непроданным.

Основные инварианты: принятая ставка соблюдает шаг и бюджет, у лота не более одного лидера и окончательного победителя, закрытый лот не принимает ставки. Проверка срока и изменение состояния выполняются как одна последовательная операция, поэтому ставка однозначно упорядочена относительно закрытия.

2. Интерфейс, данные и запуск

Интерфейс консольный: параметры разбираются через getopt, события записываются в терминал и журнал через write. Вывод показывает открытие и закрытие лотов, ставки, лидеров, резервы, продления, победителей, оплаты и итоговые бюджеты. В варианте с общей памятью независимые процессы выводят свои события, а в варианте с очередями весь вывод ведёт аукционист.

Все команды выполняются в Linux из каталога hw3. Нужны компилятор C, make и Python 3 для тестов. Без -f используются встроенные данные, с -f - указанный файл.

```
make
./auction_named -f tests/data/simple.txt -l named.log -d 100
./auction_unnamed -f tests/data/simple.txt -l unnamed.log -d 100
```

Независимый вариант: сначала аукционист, затем после сообщения «Ожидание процессов» - рабочие в двух других терминалах:

```
./auctioneer -n demo -f tests/data/simple.txt -w 300 -l shared.log
./participant -n demo -i 1
./lot -n demo -i 1
```

Для очередей используются команды auctioneer_mq, participant_mq и lot_mq с теми же аргументами. Для N участников и M лотов запускаются все N + M рабочих; номера начинаются с 1, имя сеанса у всех одинаковое. Торги начинаются после подключения всех процессов.

Ключ аукциониста	Назначение и значение по умолчанию
-f файл; -l файл	Входные данные и журнал. Без -f - встроенный пример; журнал - auction.log.
-d мс; -r 0 или 1	Длина такта 0-10000 мс (200; 0 означает 1 мс); правило резервов (0).
-n имя	Сеанс: 1-16 латинских букв, цифр или _. По умолчанию demo у независимых программ, имя из PID у версий с fork.
-w секунды; -h	Ожидание подключения 1-300 с (30); вывод справки.

Формат входного файла

Первая строка: число участников, лотов, одновременно открытых лотов, интервал продления, правило отказа. Далее для каждого участника: бюджет, стратегия, задержка, отказ. Затем для каждого лота: начальная цена, шаг, длительность.

```
2 1 1 2 1
100 0 1 0
150 1 2 1
20 10 5
```

Участники и лоты: 1-32; одновременно открытые лоты: 1-число лотов. Бюджет и цена: 0-1000000, шаг: 1-1000000. Длительность и задержка: 1-1000 тактов, продление: 0-1000 (0 отключает его). Стратегия 0 повышает на шаг, 1 - на два. Правило отказа 1 разрешает отказ; признак отказа участника 1 включает его. В остальных случаях значения равны 0.

Проверяются диапазоны, полнота и лишние данные. Размер файла меньше 16383 байт. Входной файл и журнал должны различаться, включая ссылки. Ввод и вывод выполняются через open, read, write и close; числа разбираются strtol.

3. Синхронизация и завершение

Разделяемая память

В первых трёх вариантах структура Auction размещается в разделяемой памяти через shm_open, ftruncate и mmap. Она содержит участников, лоты, заявки, резервы, статистику и флаги завершения. Указатели и файловые дескрипторы хранятся отдельно в каждом процессе.

Семафор lock защищает общие данные и вывод. Семафор events уведомляет аукциониста о готовой ставке или запросе закрытия. Участник отмечает готовность заявки и не меняет её до обработки. Процесс лота повторно читает срок, поэтому учитывает продление и новый раунд. Только аукционист принимает ставки и списывает деньги.

В auction_named и независимом варианте используются sem_open, sem_wait, sem_post и sem_timedwait. В auction_unnamed оба семафора находятся внутри общей памяти и создаются через sem_init с pshared = 1. Имена ресурсов зависят от сеанса: /a3_<имя>_mem, /a3_<имя>_lock, /a3_<имя>_events.

Очереди сообщений

В четвёртом варианте состояние хранится у аукциониста. Рабочие обмениваются запросами и снимками состояния через две очереди: /a3q_<имя>_in и /a3q_<имя>_out. Используются mq_open, mq_timedsend и mq_timedreceive. Запросы: JOIN - подключение, VIEW - снимок, SUBMIT - ставка, CLOSE - закрытие, STOP - остановка.

Именованный семафор /a3q_<имя>_gate защищает весь обмен «запрос - ответ», чтобы ответ получил отправитель. Аукционист этот семафор не захватывает. Участник сам выбирает лот и хранит подготовленную ставку, процесс лота проверяет срок по снимку. Устаревшая ставка повторно проверяется по текущему состоянию. Общая память для передачи данных не используется.

Время и остановка

Время модели - счётчик тактов, который аукционист увеличивает по монотонным часам. Процессы работают асинхронно, без общего пошагового барьера, с короткими паузами опроса. Порядок событий зависит от планировщика. Ставка при времени, равном сроку закрытия или большем него, отклоняется, даже если процесс лота ещё не запросил закрытие.

Штатно приложение завершается после закрытия всех лотов и расчётов. Если ставки не проходят по бюджету или сроку, лот остаётся непроданным; при отказе победителя начинается новый раунд. Каждый отказ исключает участника из данного лота, поэтому повторные открытия конечны. При некорректных входных данных программа завершается с ошибкой до начала торгов.

SIGINT (Ctrl+C) или SIGTERM любому подключённому процессу останавливает весь аукцион. Обработчик только сохраняет номер сигнала. Основной цикл отменяет незавершённые лоты, освобождает резервы и сохраняет уже оплаченные покупки.

В вариантах с fork родитель ожидает детей через waitpid. Независимые процессы подтверждают остановку через общую память или ответ очереди; ожидание ограничено пятью секундами. Владелец вызывает sem_close / sem_unlink, munmap / shm_unlink или mq_close / mq_unlink. Безымянные семафоры удаляются через sem_destroy. При ошибке запуска и тайм-ауте также предусмотрена очистка.

Коды выхода: 0 - успех, 1 - ошибка, 130 - SIGINT, 143 - SIGTERM. При одновременной ошибке и сигнале возвращается 1.

4. Результаты проверки

Проверка выполнена в локальном Linux-контейнере: Debian GNU/Linux 12, aarch64, GCC 12.2.0, GNU Make 4.3, Python 3.11.2. Команды make -j2 и make test завершились с кодом 0; все восемь программ собраны без предупреждений.

```
Все 138 проверок пройдены.
```

Скрипт tests/check.py выполняет 138 проверок: 72 сценария и 66 проверок ошибок. Каждый вариант проходит 18 сценариев; у независимых программ дополнительно проверяется тайм-аут подключения.

Сценарий	Результат во всех четырёх вариантах
simple	Лот куплен участником 1 за 15; остаток 85, резерв 0, выручка 15.
unsold, deadline	Лоты без покупателя не проданы; просроченная ставка не оплачена.
extension, reserves	Есть продление и смена лидера; оба режима резервирования проверены.
refusal, all_refuse	После отказа лот переоткрывается; в refusal покупает участник 2 за 20, при отказе всех лоты не проданы.
maximum	32 участника и 32 лота, всего 65 процессов: штатное завершение.
paid_interrupt	SIGTERM после оплаты: покупка за 15 сохранена, второй лот отменён; код 143.
SIGINT, неверный ввод	Сигнал аукционисту, участнику и лоту даёт код 130; ошибки данных и запуска - код 1.

Пример итогов simple одинаков для auction_named, auction_unnamed, auctioneer и auctioneer_mq:

```
=== ИТОГИ: завершено, t=20 ===
Лот 1: продан, победитель 1, цена 15.
Участник 1: начальный бюджет 100, потрачено 15, остаток 85, резерв 0.
Ставки: принято 1, отклонено 0. Продлений 0, отказов 0. Выручка 15.
```

Модуль tests/log_checks.py проверяет по журналу шаг, сроки, бюджеты, резервы, лидеров, продления, отказы и расчёты. Проверяются коды выхода и отсутствие ресурсов сеанса в /dev/shm и /dev/mqueue. Журналы сохранены в tests/results/, например auction_named_simple_r0.log и auctioneer_mq_paid_interrupt_r1.log.

Использование ИИ

ИИ использовался как вспомогательный инструмент при подготовке тестовых сценариев и скриптов проверки, Makefile, а также при структурировании и оформлении отчёта и README. Подготовленные материалы сопоставлены с реализацией; результаты сборки и тестирования подтверждены запуском в Linux.